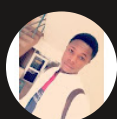
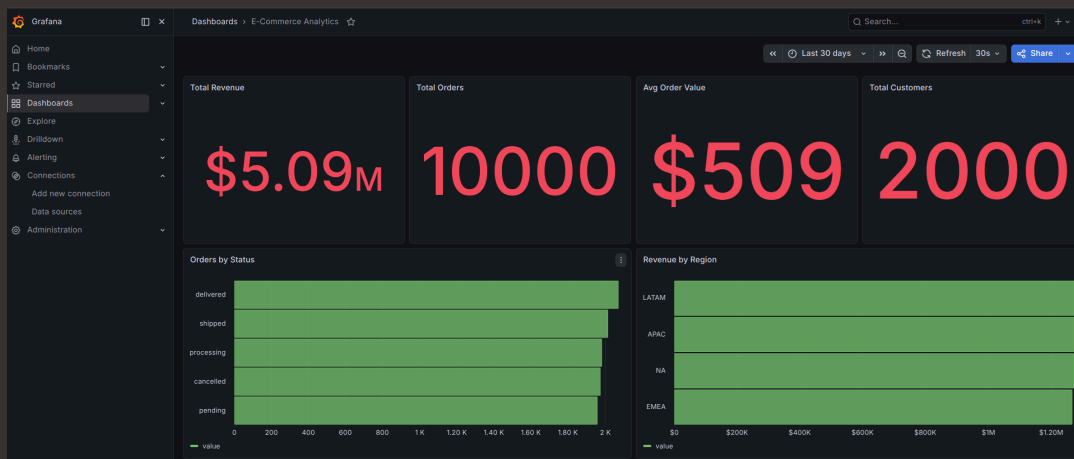


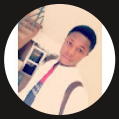


Data Visualization with Grafana MCP



Ahmed Tetteh





Introducing Today's Project!

In this project, I'm going to create a visualization dashboard. Grafana is a monitoring and visualization tool used to analyze billions of data points and transforms the SQL queries into interactive charts. I'll use it to visualize e-commerce data in real time from a PostgreSQL database.

Key tools and concepts

The key tools I used were Docker, Cursor AI, Grafana and PostgreSQL. The main concepts I learned were:

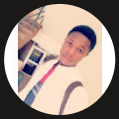
- How to create and configure Grafana dashboards
- Connecting Grafana to PostgreSQL data sources
- Building visualizations with stat panels, bar charts, and time series.
- Using MCP to manage Grafana through natural language

Challenges and wins

This project took me 1 hour 30 mins. The most challenging part was setting up the MCP servers to interact with Grafana and the PostgreSQL database. It was most rewarding to see the dashboard being updated in real-time as data kept changing in the database.

Why I did this project

I did this project because Grafana is a key tool to utilize for any monitoring workload.

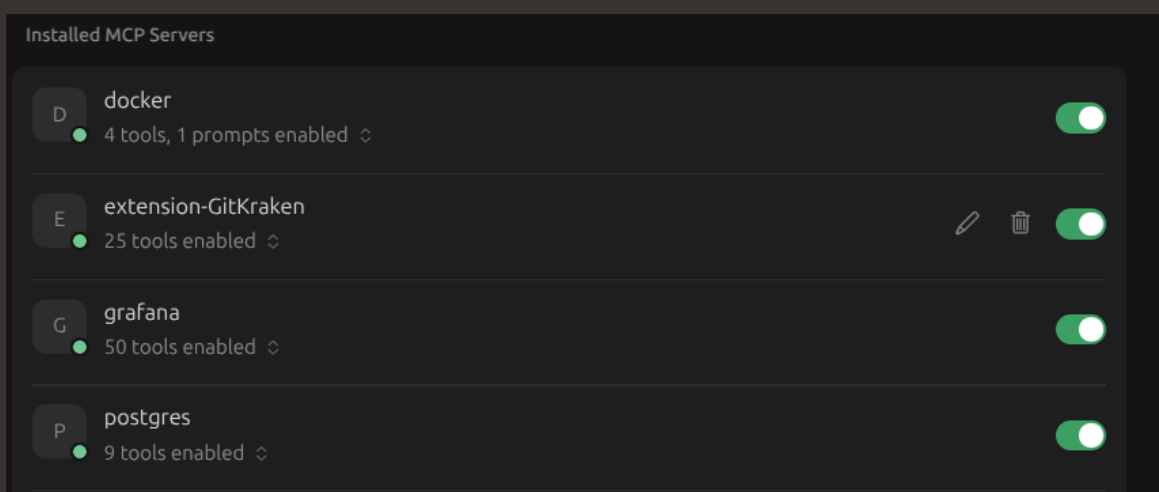


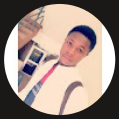
Connect Cursor to Grafana

In this step, I'm setting up the Graphana MCP to give Cursor the ability to send my commands through NLP to Graphana's API. Without an MCP server, cursor would not have the hands necessary to interact with Graphana, and I would have to manually log into Graphana, write the queries and configure everything myself.

Grafana MCP setup

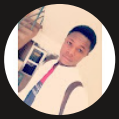
I connected Cursor to Grafana by creating an MCP server in the mcp.json file of Cursor. The Grafana MCP configuration includes the stdio,i.e, the mehtod of communication for Cursor and the MCP, the networking method, the Graphana URL and Service Access Token.





Verifying MCP connections

I verified the MCPs are connected by restarting Cursor completely. The green indicators mean that Cursor now has the ability to use Natural language Processing (NLP) in chats to execute commands within these tools. Cursor can now orchestrate these three tool (Docker, PostgreSQL and Graphana).



Creating a PostgreSQL Data Source

In this step, I'm creating a data source which is where Graphana will extract its data from to create some beatiful visualizations. This connects Grafana to the PostgreSQL database. Without this, Grafana couldn't extract the data points from the database.



Database Connection OK

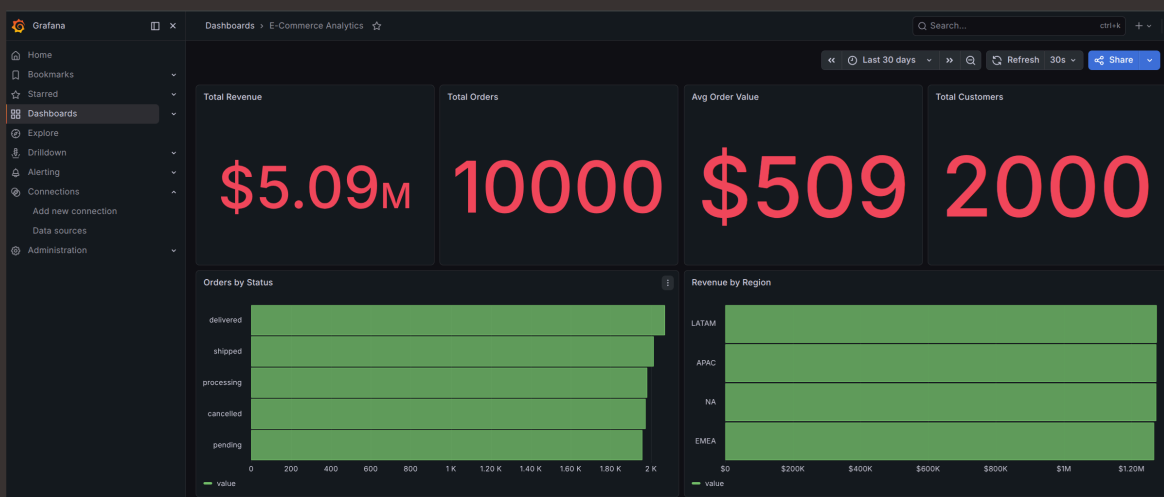
Next, you can start to visualize data by [building a dashboard from scratch](#) or by querying data in the [Explore view](#).

Data source configuration

I configured the data source with the host URL "host.docker.internal:5432" database "demo" and username "app". The connection test showed a success message. This confirms that PostgreSQL has now been set as a data source for Graphana, and Graphana knows exactly where to go to grab all the data points it needs for the visualization.

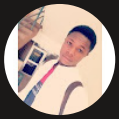
Building My First Dashboard

In this step, I'm creating a dashboard with 6 panels. Dashboards are different from SQL queries because they provide a visual interface for you glance everything at go, instead of quering numbers. The panels will show different visualization types, formats and KPIs we want to analyze.



Dashboard insights

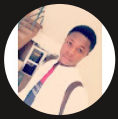
I created the dashboard by using Cursor's NLP to provide the intructions necessary to it. The top row shows metrics like otal revenue, order count, average order value, and total customers. The bottom row has bar charts showing the breakdown of orders by status, and revenue by region.



This helps me see exactly the kind of data I need in near-real time and respond to changes quicker.

Revenue trends

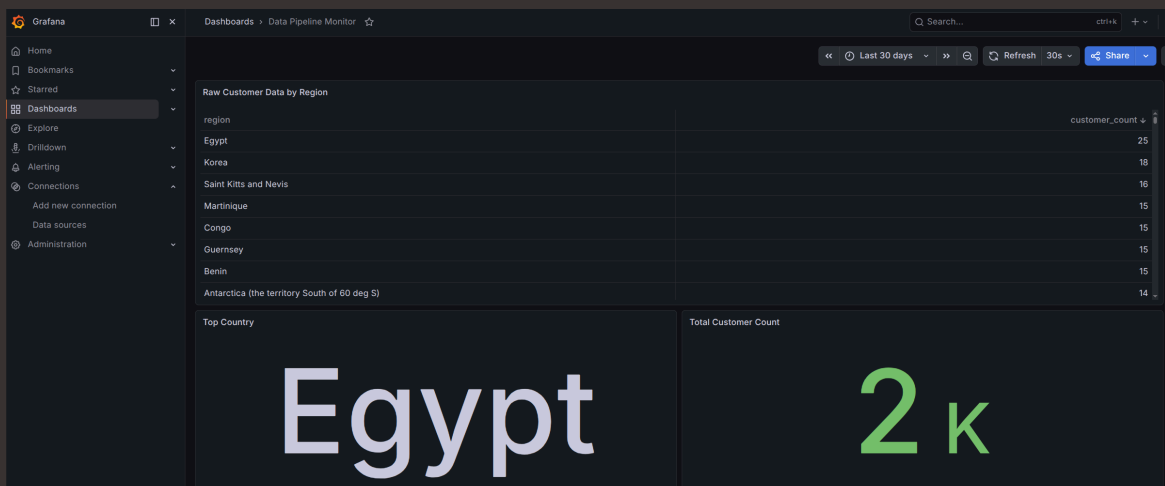
The time series chart shows the change in data over time, typically at equally spaced intervals. It's used to track how a variable changes, like stock prices, temperature, or sales figures, to identify patterns, trends, or seasonality. I can see trends like monthly highest revenue, monthly flat revenues and low month revenues. This type of visualization helps Businesses make better data-driven decisions.

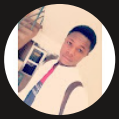


Visualizing the DBT Pipeline

Building the pipeline dashboard

I'm building a dashboard that shows the query performance between the raw and transformed data. This demonstrates the pipeline's full view by using Graphana to visualize it. A dbt model is an open-source command-line tool that helps data analysts and engineers transform data in their warehouses. It allows you to write SQL queries as models, test them, and document your data transformations





Updating the dashboard

I updated the dashboard by changing some values in my database table. I used Cursor to update the raw data and aggregated table. This aggregated or transformed table is dashboard- ready, as it helps improve the query performance. The panels refreshed and showed Panel 1 with Egypt's count increased, Panel 2 changed from Korea to Egypt as the top country, and Panel 3 will stay the same at 2K (total customer count).

Performance comparison

The difference between Panel 1 and Panels 2-3 is that Panel 1 scans all customer rows every time (slower for large datasets), while Panels 2-3 produces instant results (data already aggregated). Raw queries are not preferred for large datasets as each query scans the millions of rows everytime, making it slow for dashboards, while DBT models create materialized views (pre-computed query result tables) that pre-calculate the heavy work, making them ideal for dashboards and real-time monitoring. In production, this matters because it can help engineers spot issues much quicker, trace problems and analyze data in real-time for making better decisions.



nextwork.org

The place to learn & showcase your skills

Check out nextwork.org for more projects

